

```
"""
```

Bid Intelligence – Streamlit web app

Drag-drop tender PDFs, get a structured CSV/Excel back.

Run locally:

```
pip install -r requirements.txt
streamlit run app.py
```

Deploy free on Streamlit Cloud:

1. Push this file + requirements.txt to a GitHub repo
2. Go to <https://share.streamlit.io>, sign in, "New app"
3. Pick the repo, set main file to app.py
4. Under "Advanced settings → Secrets", add:

```
ANTHROPIC_API_KEY = "sk-ant-..."
```

5. Deploy. You'll get a public URL like
<https://your-app-name.streamlit.app>

```
"""
```

```
from __future__ import annotations
```

```
import io
```

```
import json
```

```
from typing import Optional
```

```
import pandas as pd
```

```
import pdfplumber
```

```
import streamlit as st
```

```
from anthropic import Anthropic
```

```
from pydantic import BaseModel, Field, ValidationError
```

```
# ----- Schema -----
```

```
class FinancialPQ(BaseModel):
```

```
    min_turnover_inr_cr: Optional[float] = None
```

```
    min_net_worth_inr_cr: Optional[float] = None
```

```
    similar_work_value_inr_cr: Optional[float] = None
```

```
    notes: Optional[str] = None
```

```
class TechnicalPQ(BaseModel):
```

```
    similar_projects_required: Optional[int] = None
```

```
    min_capacity_mld: Optional[float] = None
```

```

    notes: Optional[str] = None

class TenderRecord(BaseModel):
    tender_id: Optional[str] = None
    issuing_authority: str
    title: str
    location_state: Optional[str] = None
    location_city: Optional[str] = None
    sector: str = Field(description="water_supply | wastewater | reuse | desal |
contract_model: str = Field(description="EPC | PPP | HAM | BOT | OM | mixed")
capacity_mld: Optional[float] = None
contract_tenure_years: Optional[float] = None
estimated_project_cost_inr_cr: Optional[float] = None
emd_inr_cr: Optional[float] = None
funding_source: Optional[str] = None
submission_deadline: Optional[str] = None
bid_validity_days: Optional[int] = None
technical_pq: TechnicalPQ = Field(default_factory=TechnicalPQ)
financial_pq: FinancialPQ = Field(default_factory=FinancialPQ)
key_clauses_to_review: list[str] = Field(default_factory=list)
summary: str

# ----- Extraction -----

EXTRACTION_PROMPT = """You are an analyst at an Indian water-infrastructure compa
reviewing an RFP/tender document. Extract the structured fields described in the
JSON schema.

Rules:
- Output ONLY a single JSON object. No prose, no markdown fences.
- If a field is genuinely not in the document, use null.
- Convert all monetary figures to crore INR (1 cr = 10,000,000 INR).
- Convert capacities to MLD (1 MGD ≈ 4.546 MLD; 1000 m3/d = 1 MLD).
- For key_clauses_to_review, surface non-standard or risky clauses
  (liquidated damages, escalation, change-in-law, force majeure, payment
  peculiarities). Skip standard boilerplate.
- For summary, 3-4 sentences in plain English.

JSON schema:
{schema}

Document follows.
=====
{rfp_text}
=====

```

```
"""
```

```
def extract_pdf_text(file_bytes: bytes) -> str:
    chunks: list[str] = []
    with pdfplumber.open(io.BytesIO(file_bytes)) as pdf:
        for i, page in enumerate(pdf.pages):
            text = page.extract_text() or ""
            chunks.append(f"\n--- Page {i + 1} ---\n{text}")
    return "\n".join(chunks)

def extract_with_claude(rfp_text: str, client: Anthropic) -> TenderRecord:
    schema_json = json.dumps(TenderRecord.model_json_schema(), indent=2)
    body = rfp_text[:120_000] # truncate; smarter chunking is a v2 problem
    resp = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=4096,
        messages=[{
            "role": "user",
            "content": EXTRACTION_PROMPT.format(schema=schema_json, rfp_text=body
        )}],
    )
    raw = resp.content[0].text.strip()
    if raw.startswith("`"):
        raw = raw.split("`", 2)[1]
        if raw.lstrip().startswith("json"):
            raw = raw.split("\n", 1)[1]
    data = json.loads(raw)
    return TenderRecord(**data)

def flatten_for_spreadsheet(record: TenderRecord, source_file: str) -> dict:
    """Pydantic record → flat dict suitable for a CSV row."""
    d = record.model_dump()
    tech = d.pop("technical_pq") or {}
    fin = d.pop("financial_pq") or {}
    clauses = d.pop("key_clauses_to_review") or []
    d["technical_pq_min_capacity_mld"] = tech.get("min_capacity_mld")
    d["technical_pq_similar_projects"] = tech.get("similar_projects_required")
    d["technical_pq_notes"] = tech.get("notes")
    d["financial_pq_turnover_cr"] = fin.get("min_turnover_inr_cr")
    d["financial_pq_networth_cr"] = fin.get("min_net_worth_inr_cr")
    d["financial_pq_similar_work_cr"] = fin.get("similar_work_value_inr_cr")
    d["financial_pq_notes"] = fin.get("notes")
    d["key_clauses"] = " | ".join(clauses)
    d["source_file"] = source_file
```

```

return d

# ----- UI -----

st.set_page_config(page_title="Water Sector Bid Extractor", page_icon="📄", layout="wide")
st.title("📄 Water Sector Bid Extractor")
st.caption("Upload tender PDFs → get a structured spreadsheet of bid metadata.")

# API key: prefer Streamlit secrets (production), fall back to user input (local/development)
api_key: Optional[str] = None
try:
    api_key = st.secrets["ANTHROPIC_API_KEY"]
except (KeyError, FileNotFoundError, st.errors.StreamlitSecretNotFoundError): #
    pass
if not api_key:
    api_key = st.text_input("Anthropic API key", type="password",
                           help="Get one at console.anthropic.com. Or set ANTHRO")

uploaded = st.file_uploader(
    "Drag in tender PDFs",
    type=["pdf"],
    accept_multiple_files=True,
)

run = st.button("Extract", type="primary", disabled=not (api_key and uploaded))

if run:
    client = Anthropic(api_key=api_key)
    rows: list[dict] = []
    errors: list[str] = []
    progress = st.progress(0.0)
    status = st.empty()

    for i, f in enumerate(uploaded):
        status.text(f"[{i + 1}/{len(uploaded)}] {f.name}")
        try:
            text = extract_pdf_text(f.read())
            record = extract_with_claude(text, client)
            rows.append(flatten_for_spreadsheet(record, f.name))
        except (ValidationError, json.JSONDecodeError) as e:
            errors.append(f"{f.name}: schema/JSON error - {e}")
        except Exception as e:
            errors.append(f"{f.name}: {type(e).__name__} - {e}")
        progress.progress((i + 1) / len(uploaded))

    status.empty()

```

```

progress.empty()

if errors:
    with st.expander(f"⚠ {len(errors)} file(s) failed"):
        for err in errors:
            st.write(err)

if rows:
    df = pd.DataFrame(rows)
    st.success(f"Extracted {len(rows)} tender(s).")
    st.dataframe(df, use_container_width=True)

    csv_bytes = df.to_csv(index=False).encode("utf-8")
    st.download_button("⬇ Download CSV", csv_bytes, "tenders.csv", "text/csv")

    excel_buf = io.BytesIO()
    with pd.ExcelWriter(excel_buf, engine="openpyxl") as writer:
        df.to_excel(writer, index=False, sheet_name="Tenders")
    st.download_button(
        "⬇ Download Excel",
        excel_buf.getvalue(),
        "tenders.xlsx",
        "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    )

```